

Scaling Semantic Models the Smart Way: Governance, Automation, and Reuse

- **Product Evangelist at Tabular Editor**

- Pre-sales, events, marketing
- Client training

- **Microsoft Data Platform MVP**

- **Found and organizer of**

- Danish Microsoft BI Community (MsBIP.dk)
- Power BI UG Denmark (PowerBI.dk)
- Data Platform Next Step, Power BI Next Step and Power BI Cruise



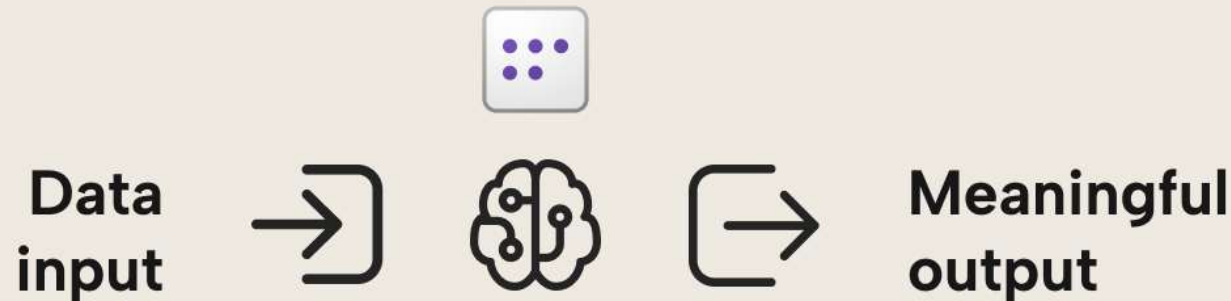
[linkedin.com/in/blindbaek/](https://www.linkedin.com/in/blindbaek/)

Introduction

- Semantic models are becoming the **most important data asset** in Power BI / Fabric
 - Semantic models add **business meaning** to data
 - Necessary for **both humans and AI agents** to make sense of the data in a **business context**
 - In many ways, it is the **central nervous system** of your data estate

"Semantic models aren't just a powerful, re-usable toolbox; they are strategic assets critical for an organization to get the most from data, analytics, and AI (including LLMs and AI agents). Good semantic models can help you create good reports, analyses, and tools, which in turn can help you achieve organizational business objectives."

- Kurt Buhler, <https://tabulareditor.com/blog/semantic-models-the-brain-for-analytics-reporting-and-ai-with-data>



...but it's not as simple as it sounds

- M and DAX?!?
- Refresh-schedules and query performance, sizing and scaling
- Documentation?
- Source-control integration, testing, multiple environments, automated deployments?
- Governance, adoption, security?

Key challenges that need to be kept top of mind:

- Balancing agility and maintainability as models scale in size and number
- Ensuring correctness, consistency and “single source of truth”

The Core Dilemma: Fewer Larger Models vs. Many Smaller Models

Fewer Larger Models (monolithic)

Pros:

- Centralized governance, single source of truth
- Easier cross-domain measures (e.g. enterprise KPIs)

Cons:

- Risk of performance bottlenecks
- Slower change cycles, potential for dependency conflicts

Many Smaller Models (modular/domain-based)

Pros:

- Faster iteration, domain ownership, easier testing
- Natural fit for data mesh or departmental setups

Cons:

- Duplicate logic, inconsistent definitions
- Harder to manage shared time intelligence and KPIs

Power BI / Fabric / semantic model features that enable reuse

- Existing features focus mostly on logic reuse within the context of a single model:
 - User-Defined Functions (UDFs)
 - Calendar-based Time Intelligence
 - Calculation Groups
- Some features are useful when working with multiple models:
 - Composite Models (aka. “Direct Query over Analysis Services and Power BI semantic models”, or just “DQ over AS”)
 - Metric Sets (Deprecated as of October 2025 😞)
 - Alternative: Power BI scorecards
- But ultimate control and flexibility requires custom solutions:
 - “Master Model Pattern”

User-Defined Functions (UDFs)

- Preview feature as of September 2025
- Only available in Power BI / Fabric (Compatibility Level 1702 – not available on SSAS / Azure AS)
- Two types of UDFs:
 - **Model dependent:** References specific model objects (measures, columns, tables). Encapsulate / reuse logic inside a single model. Examples: Currency conversion using exchange rate data, time intelligence.
 - **Model independent:** Does not reference any model objects. Encapsulate logic that can be reused across multiple models. Examples: SVG measures, format strings, unit conversions.
- No 1st party solutions to host central UDF repository
 - Custom solutions / scripts to the rescue!

Demo

User-Defined Functions

Calendar-based Time Intelligence

- Preview feature as of September 2025
- Only available in Power BI / Fabric (Compatibility Level 1701 – not available on SSAS / Azure AS)
- Not specifically a feature that emphasizes logic reuse
 - Calendar-based Time Intelligence *simplifies* time intelligence calculations
 - Less DAX, less code that must be reused, smaller risk of inconsistencies
 - Plays nicely with script-based reuse solutions, as your date table encapsulates the calendars / time units of your model

Demo

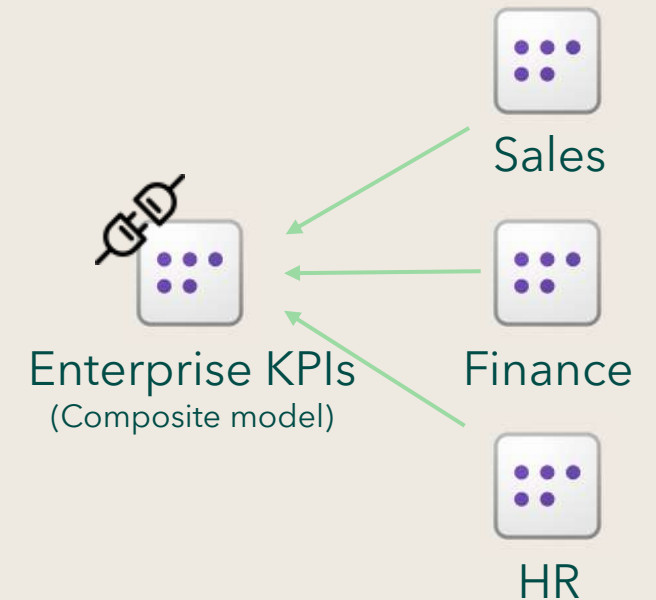
Calendar-based Time Intelligence

Calculation Groups

- The go-to solution for logic reuse before UDFs
- Still viable for certain scenarios, when:
 - You need logic that impacts all or most measures of your model
 - You need logic that changes based on a user selection in a slicer
- Has certain drawbacks
 - Performance implications
 - Complexity, difficult to debug
- Also doesn't enable reuse across models
 - ...still requires custom scripting solutions

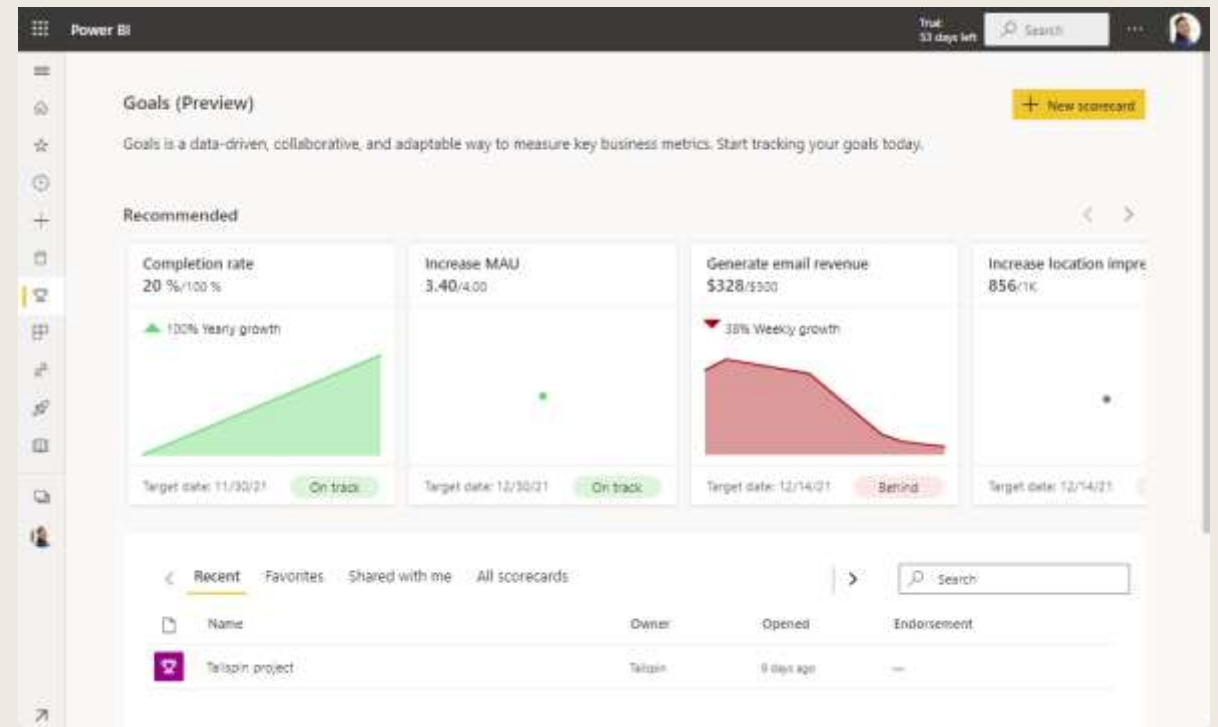
Composite Models ("DQ over AS")

- Semantic models that will "pass through" queries to one or more underlying models, combining the data on the fly
- Multiple use cases
 - Enterprise reporting across multiple domain specific models
 - Enrich existing models (e.g. mashup an existing model with budget numbers from an Excel file)
 - Rapidly prototype and test new model features
- Best practices:
 - Keep logic in source models, minimize logic in composite model
 - Avoid expensive relationships (i.e. avoid relationships across model boundaries, when there are many rows in the common dimension)



Power BI scorecards

- Simpler solution than composite model for cross-model dashboard creation, but...
 - Much less flexible
 - No options to create regular reports
 - Definitions live “outside” the semantic models, maintenance overhead



So what else can we do?

- Semantic model objects can be expressed as **code** (TMSL / TMDL)
- Semantic models have a rich programming model called TOM
 - Accessible through PowerShell, .NET, Python (Semantic Link)
 - Used by 3rd party tools (Tabular Editor, Bravo, and others)
- TOM metadata can be accessed **from files**, or through the Fabric **REST API**, or through the **XMLA endpoint**
- This is a huge invitation for tailor-made, custom solutions, “polyfilling” all areas where the built-in features are insufficient:
 - Logic reuse
 - Ensuring consistency
 - Automated testing
 - Deployments
 - Etc.

Introducing the “Master Model Pattern”

- One single model as the “source of truth”, aka. the “Master Model”
 - Only exists as model metadata – lives in your git repo and never actually deployed anywhere
 - Accessed by all model developers and maintainers
- Leverages the “Perspectives” feature to define “sub-models”
 - Sub-models typically have domain-specific / departmental focus
 - Sub-models get deployed where they are needed
- Scripts are used to produce (“build”) the sub-models from the master model
- Easily manageable in Tabular Editor but the master model can also be edited in Power BI Desktop

Demo

Master Model Pattern

Alternative scripting technique for inspiration

- **Modular approach:**

- Maintain commonly used model objects (measures, date tables and other common dimensions, etc.) in a central repo using TMDL
- Use a script to “load” the central definitions into the model you’re currently working with
- Script can be run ad-hoc, or as part of automated deployments
- Use annotations for tracking, versioning, etc.
- If you’re not comfortable with scripting, start slow:
 - Copy/paste objects between instances of Tabular Editor
 - Copy/paste TMDL between folder structures
 - Once you start to see emerging patterns of reuse – ask your favorite AI to help you automate it

Summing up

- There's no one-size-fits-all design – governance and reuse are the real differentiators.
- Modern features (UDFs, time intelligence) make modularity feasible without duplication, in the context of a single model
- But when dealing with multiple models, we need something more powerful
- Tabular Editor C# scripting has existed for almost 10 years – it was the first tool to bring software engineering disciplines to semantic modelling in the Microsoft world
 - Many programmability options exist today – choose the one you're most comfortable with!
- As our semantic models scale, it's not about choosing big or small – **it's about designing smart.**

Next step: Optimizing Semantic Model Size

- Analyze memory consumption with VertiPaq Analyzer
- Apply automatic and custom aggregations
- Understand SKU memory limits and how to design within them
- Optimize memory during refresh
 - Implement incremental refresh patterns effectively
 - Use hybrid tables to balance freshness and performance
 - Take advantage of Direct Lake mode where it makes sense
 - Apply custom partitioning for more control